

Analyseur de langage clair — aide-mémoire

Louis Hubeaux · Python / Flask
État au gel du 23/09, réverifié le 27/09

En une phrase : une application Flask qui reçoit un texte administratif en français et **montre dans le texte** ce qui l'éloigne des dix principes de rédaction claire de la Commission européenne. Chaque signalement a une position exacte, un principe et, si possible, une reformulation. **Pas de note sur 100** : un score fait optimiser un chiffre, un surlignage fait relire.

3 règles : longueur (P4), connecteurs (P5), passif (P8)	48 tests + 1 xfail, en ~3 s	4/6 passif sur les phrases de référence (2/6 avec les dépendances seules)	25 mots : seuil de phrase en français (21 en anglais)
16 · 17 connecteurs lourds · verbes avec <i>être</i>	4 formats : .txt .md .docx .odt (.pdf et .doc refusés)	8 routes, dans deux blueprints (+ /health)	5 tables SQLite, 3 migrations

DÉROULÉ DES 21 DIAPOS — LE MESSAGE DE CHACUNE

1	Titre : laisser la phrase à l'écran, on y revient à la fin.
2	Qui je suis : 1 min 15 max, finir sur le lien avec le sujet.
3	Problème : correct ≠ lisible (44 mots, 6 locutions).
4	Sujet : l'outil montre l'endroit du problème.
5–6	Référentiel : 3 principes détectés ; 2 pistes ; 5 hors de portée d'une machine.
7	Public : agents publics, juristes. Pas de score : décision assumée.
8	Démonstration du parcours complet (≈ 5 min).
9	Stack : citer 2 motifs (pysbd, spaCy), pas le tableau.
10	Chaîne : un seul point d'entrée, <code>build_document()</code> .
11	Architecture : un dossier, une responsabilité ; deux registres.
12	Règle d'or : <code>texte[start:end] == fragment</code> .
13	Normaliser : six transformations, une seule fois (p. 2).
14	Segmenter : paragraphes puis phrases ; 285 + 49 (p. 2).
15	Tokeniser : 285 + 36 = 321 — des caractères, pas des mots (p. 3).
16	Passif : même forme, quatre constructions ; une regex échoue.
17–18	spaCy : l'arbre de dépendances, puis 5 questions posées à l'arbre.
19	Heuristiques ① cop ② VERB ③ verbes avec <i>être</i> : 2/6 → 4/6.
20	Bilan : fait, à faire, refusé ; gel le 23/09.
21	Fin : « Pour assurer le suivi, le service publie une notice. »

COMMANDES

```
.venv\Scripts\Activate.ps1
flask db upgrade          # la structure : tables
flask seed                # le contenu : lexiques.json → base
flask run                 # http://127.0.0.1:5000
flask routes              # la table des routes, p. 4
.venv\Scripts\python.exe -m pytest -p no:cacheprovider
# une seule suite, en direct :
... -m pytest -p no:cacheprovider tests/test_passif.py -v
```

Lanceur cassé → `.venv\Scripts\python.exe -m flask run`. Script d'activation refusé → `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`. **Ne pas recréer le venv** : `requirements.txt` n'épingle pas les versions.

DÉMONSTRATION — CHECK-LIST

- ☐ `.venv\Scripts\Activate.ps1` puis `flask run`
- ☐ Base à jour : `flask db upgrade` puis `flask seed`
- ☐ Une analyse à blanc : spaCy se charge (~2 s)
- ☐ « à cet égard » **absent** de la liste (sinon le supprimer)
- ☐ Dossier `exemples/` ouvert, un `.pdf` sous la main
- ☐ Terminal prêt pour `pytest` ; zoom navigateur 125 %

Le parcours (diapo 8)

- 1 Analyser** les deux paragraphes : les trois règles se déclenchent ; longueur et connecteurs se superposent sans <mark> imbriqués.
- 2 Cliquer un surlignage** : la fiche s'ouvre (principe, position, proposition).
- 3 Diagnostic technique** (sous le résultat) : § 2 à 285–548, 1^{re} phrase à 285–334. Les chiffres des pages 2 et 3, en direct.
- 4 Listes de mots** : coller « À cet égard, le dossier est complet. » → rien. Ajouter « à cet égard » → « sur ce point ». Relancer → signalé. Aucun redémarrage.
- 5 Enregistrer** sous un nom → redirection → *Analyses enregistrées* → rouvrir : surlignage identique. La base ne stocke **aucun HTML**.
- 6 Importer** `notification.docx` (mêmes signalements que les 3 autres formats), puis un **.pdf** : message dédié, pas « format inconnu ».

Le texte de démo : docs/soutenance.md, § 6

```
§ 1 la phrase de 44 mots          # P4 + P5 superposés
§ 2 « une notice est publiée »    # sans agent → avertissement
  « est rejetée par le biais... » # avec agent → info
```

Plan B : la capture de la diapo 8, puis `pytest tests/test_passif.py -v`. Réverifié le 27/09 sur la vraie base : 12 connecteurs, 1 phrase longue, 2 passifs (321–332 et 504–515).

GARDE-FOUS ET RÉGLAGES

MAX_TEXT_LENGTH	20 000 caractères, vérifié côté serveur
MAX_CONTENT_LENGTH	2 Mo ; au-delà : 413, même page, message dédié
MAX_FORM_MEMORY_SIZE	aligné sur 2 Mo : un texte trop long reçoit son propre message
NOM_MAX	100 caractères pour le nom d'une analyse
SECRET_KEY	signe le cookie de session, donc les messages flash
Modèle spaCy	<code>fr_core_news_sm</code> 3.8.0, épinglé par URL
Réseau	aucune requête à l'exécution ; listes dans <code>data/seeds/lexiques.json</code>

ORDRE DE SACRIFICE

conteneurisation → export → anglais → administration → règles supplémentaires → tokenisation fine. **Jamais** : la chaîne, les trois règles, le surlignage, le passif, les tests, la répétition.

LA RÈGLE D'OR : L'INVARIANT DES POSITIONS

```
document.texte[start:end] == fragment.texte # partout
```

Vrai pour chaque paragraphe, phrase, token, token spaCy et signalement, et vérifié par tests/test_positions.py. Chaque couche **transporte** sa position par **addition**, elle ne la recherche jamais.

Les deux pièges : texte.find() retombe sur la 1^{re} occurrence d'une phrase répétée ; cumuler des longueurs de paragraphes dérive à chaque ligne vide.

① NORMALISER — SIX TRANSFORMATIONS, UNE SEULE FOIS

Quoi	Devient	Pourquoi
BOM en tête (U+FEFF)	retiré	marque invisible laissée par certains éditeurs
\r\n et \r	\n	sinon chaque ligne Windows décale tout d'un caractère
NFC	e + ó → é	« é » compte 1 caractère, pas 2
césure conditionnelle (U+00AD)	supprimée	invisible, mais coupe un mot pour les regex
apostrophe courbe ’	’	les lexiques n'ont qu'une orthographe à connaître
insécables (U+00A0, U+202F)	espace	idem : une seule espace possible

LA CHAÎNE, DANS L'ORDRE

```
saisie, ou fichier → extraire() # texte brut
build_document(texte_brut, langue="fr")
    normalize() # une seule fois
    segment() # paragraphes → phrases
    tokenize() # \S+ avec offset
    analyser_phrases() # spaCy, phrase par phrase
run(document) # REGLES → Finding, triés
surligner() # échappe, puis <mark>
```

Pourquoi une seule fois, et au début ? Quatre de ces six transformations **raccourcissent** le texte. Une position calculée avant serait faussée après. build_document() normalise donc en premier et nulle part ailleurs ; le texte normalisé devient **la** référence, celle qu'on réaffiche au lecteur.

Idempotente : normalize(normalize(t)) == normalize(t). C'est ce qui rend la relecture d'une analyse exacte (p. 4), et c'est la même fonction qui nettoie les saisies de l'écran des listes.

Un extracteur ne normalise jamais : sinon texte collé et fichier importé auraient deux textes de référence.

② SEGMENTER — OÙ L'ON COUPE, EN FRANÇAIS

PITCH · 30 SECONDES

« On découpe en deux temps. D'abord les **paragraphes** : une ligne vide sépare deux paragraphes, un simple retour à la ligne non. Ensuite, dans chaque paragraphe, les **phrases**, avec pysbd, une bibliothèque qui connaît les usages du français : elle coupe après un point, un point d'exclamation, un point d'interrogation ou des points de suspension quand une nouvelle phrase commence, mais pas après « M. » ou « etc. », ni dans « 3.5 » ou « C.E.E. ». On ne donne jamais le document entier à pysbd, seulement un paragraphe. Il rend la position de la phrase *dans* le paragraphe, et on y ajoute la position du paragraphe *dans* le texte : 285 + 0 = 285. »

Ce que pysbd rate (mesuré le 27/09)

Il coupe à tort après « **l'art.** » (l'élision masque l'abréviation), « janv. 2026 », « Réf. », « Pr. », « p. ex. », « Mme. ». **Effet** : une phrase coupée en deux, donc plus courte. Au pire, la règle de longueur manque une phrase. Jamais de faux signalement ni de position faussée.

Où tombent les positions

L'espace qui suit le point **reste à la phrase qu'elle termine** : dans un paragraphe, les phrases se touchent bout à bout (le end de l'une est le start de la suivante).

```
§ 1      0 → 283 # la phrase de 44 mots
  \n\n    283 → 285 # n'appartient à personne
§ 2      285 → 548 # 3 phrases
  phrase 1 285 → 334 # 285 + 49, espace finale comprise
  phrase 2 334 → 464
  phrase 3 464 → 548
```

```
bloc.start() + span.start # 285 + 0 = 285
# position du paragraphe + position dans le paragraphe
```

Deux réglages de pysbd : clean=False lui interdit de nettoyer le texte (on veut ses positions, pas sa réécriture) ; char_span=True lui fait rendre des positions de caractères.

Pourquoi pas spaCy pour découper ? Mesuré : il fusionne un titre avec la phrase suivante. pysbd découpe, spaCy analyse des phrases déjà isolées. Il n'y a qu'une seule définition de « phrase » dans le projet.

Niveau 1 — les paragraphes (notre code)

- ✂ à chaque **ligne vide** (deux retours à la ligne ou plus).
- pas à un retour à la ligne simple : les lignes qui se suivent restent dans le même paragraphe ;
- un bloc fait seulement d'espaces est ignoré.

Niveau 2 — les phrases (pysbd, langue « fr »)

- On coupe :
- ✂ après . ! ? ... suivis d'une espace et d'une nouvelle phrase : « Est-ce complet ? Oui ! » → 2 phrases ;
 - ✂ à un **retour à la ligne simple** : un titre sans point (« Objet de la demande ») ou chaque ligne d'une liste à puces devient sa propre phrase.
- On ne coupe pas :
- après une abréviation connue suivie d'un mot : *M., Mme, etc., cf., art.* (« selon art. 12 ») ;
 - dans un nombre (3.5), un sigle à points (C.E.E.), une adresse (*www.europa.eu*) ;
 - entre guillemets « ... » ou entre parenthèses, même s'il y a un point à l'intérieur.

③ TOKENISER — UN MOT, UNE POSITION ABSOLUE

PITCH · 30 SECONDES

« Un token, c'est un mot avec deux nombres, start et end. Ce sont ses positions **en caractères** dans le texte normalisé, pas des numéros de mots. La phrase « Afin d'assurer le suivi, une notice est publiée. » commence au caractère 285. Dans la phrase, « est » se trouve 36 caractères plus loin : **285 + 36 = 321**. « est publiée » fait 11 caractères, donc l'empan se termine à **332**, borne exclue comme en Python. Un test vérifie que `texte[321:332] == "est publiée"`. La position n'est jamais cherchée, elle est additionnée. C'est pour ça que le surlignage tombe toujours au bon endroit. »

```
tokenize(phrase["texte"], phrase["start"])
# re.finditer(r"\S+") : un token = une suite sans espace
start = offset + match.start()      # 285 + 36 = 321
```

n°	token	start	end
0	Afin	285	289
1	d'assurer	290	299
...	...	...	...
5	notice	314	320
6	est	321	324
7	publiée.	325	333

Les **numéros** de token sont 6 et 7 ; les **positions** vont de 321 à 332. Ne pas dire « tokens 321 à 332 ».

Deux tokenisations, deux usages

	Sentence.tokens (nous)	Sentence.analyse (spaCy)
découpe	sur les espaces	règles du français : élision, ponctuation
la phrase	8 tokens	11 : Afin · d' · assurer · le · suivi · , · une · notice · est · publiée · .
porte	texte, start, end	+ lemme, catégorie, fonction, gouverneur
sert à	compter les mots (longueur)	la grammaire (passif)
position	offset + match.start()	offset + token.idx

connecteurs_lourds n'utilise ni l'une ni l'autre : sa regex balaie document.texte lui-même, et ses positions sont donc absolues par construction.

Les questions pièges

Pourquoi 332 et pas 333 ?

Avec \S+, le point reste collé (« publiée. » finit à 333). L'empan du passif est construit à partir des tokens spaCy, qui séparent la ponctuation : il va du premier auxiliaire (321) à la fin du participe (325 + 7 = 332).

D'où vient 285 ?

Le § 1 fait 283 caractères, suivis de deux \n.

« La décision a été prise. » : combien de mots ?

5 pour la règle de longueur, 6 tokens pour spaCy (le point).

Pourquoi une tokenisation aussi grossière ?

Elle suffit pour compter. L'analyse fine vient de spaCy, à part, et version_tokeniseur en base protège les anciennes positions le jour où on l'affinera.

④ ANALYSER AVEC SPACY — LA MÊME PHRASE, VUE PAR L'ARBRE

```
analyser_phrases([(phrase["texte"], phrase["start"]), ...])
# les phrases de pysbd, en un lot : modele().pipe()
start = offset + token.idx      # la même addition
```

Chaque token spaCy est recopié dans un TokenLinguistique maison, une dataclass figée : texte, lemme, categorie (pos_), fonction (dep_), gouverneur (head.i), start, end. Les règles ne voient jamais spaCy : seul linguistics.py l'importe.

Une phrase à la fois : gouverneur est un index *dans la phrase*, d'où phrase.analyse[auxiliaire.gouverneur]. Le passif lit donc l'arbre sans rien rechercher dans le texte.

Les cinq questions, sur « une notice est publiée »

1 « est » : fonction aux:pass → c'est un auxiliaire de passif. **2** Son gouverneur, le token 9, c'est « publiée ». **3** Catégorie VERB ✓ (pas un attribut). **4** Lemme *publier*, absent des 17 verbes avec *être* ✓. **5** Aucun obl:agent → **avertissement**. L'empan va du premier auxiliaire à la fin du participe : **321 → 332**.

n°	texte	lemme	catégorie	fonction	gouv.	start–end
0	Afin	afin	ADV	advmod	9	285–289
1	d'	de	ADP	case	2	290–292
2	assurer	assurer	VERB	obl:arg	0	292–299
3	le	le	DET	det	4	300–302
4	suivi	suivi	NOUN	obj	2	303–308
5	,	,	PUNCT	punct	9	308–309
6	une	un	DET	det	7	310–313
7	notice	notice	NOUN	nsubj:pass	9	314–320
8	est	être	AUX	aux:pass	9	321–324
9	publiée	publier	VERB	ROOT	9	325–332
10	.	.	PUNCT	punct	9	332–333

Sortie réelle de `analyser_phrases()` avec `fr_core_news_sm 3.8.0`. Le token 9 est la racine (ROOT) : il est son propre gouverneur.

⑤ SIGNALER — LES TROIS RÈGLES LIVRÉES

Règle	Principe · sévérité	Lit	Mécanisme
longueur_phrase	P4 · avertissement	Sentence.tokens	compte les tokens de chaque phrase et compare au seuil de la langue (25) ; l'empan est la phrase entière
connecteurs_lourds	P5 · info	document.texte	16 locutions en base avec leur reformulation. Motif tolérant : « afin de » → \bafin\s+d(?:e\b '), sans tenir compte de la casse, plusieurs espaces admises, élision « d' ». Les plus longues d'abord ; les caractères déjà pris sont écartés, donc pas de double signalement
passif	P8 · info / avert.	Sentence.analyse	les cinq questions ci-dessus ; avec un agent → info, sans agent → avertissement

Chaque règle renvoie des Finding : rule_id, hint (le principe), severity, char_start, char_end, message, suggestion. Les positions sont toujours absolues dans document.texte.

LES ROUTES — FLASK ROUTES LES AFFICHE

Méthode	URL	Fonction	Ce qu'elle fait	Répond
GET	/	analyze.index	formulaire vide	200
POST	/	analyze.index	analyse le fichier ou le texte ; n'enregistre rien	200
GET	/analyses/	analyze.analyses	la liste, la plus récente d'abord	200
POST	/analyses/	analyze.enregistrer	relance l'analyse, l'enregistre sous un nom	302 ; 400
GET	/analyses/<int:analysis_id>	analyze.relire	relit, sans relancer les règles	200 ; 404
GET	/listes/	admin.listes	affiche les listes de mots	200
POST	/listes/<int:liste_id>/entrees	admin.ajouter	ajoute une entrée (normalisée, sans doublon)	302 ; 404
POST	/listes/entrees/<int:entree_id>/supprimer	admin.supprimer	supprime une entrée	302 ; 404
GET	/health	health	{"status": "ok"}	200

Deux blueprints : analyze sans préfixe, admin avec url_prefix="/listes" ; /health est déclarée dans create_app(). Plus /static/... et app_errorhandler(413), qui rend la même page via page(). Le convertisseur <int:...> fait qu'une URL comme /analyses/abc reçoit un 404 sans que la fonction soit appelée. Les URL se construisent par le nom de la fonction : url_for("analyze.relire", analysis_id=...).

LE PARCOURS D'UN POST /

- 0 Avant la route : au-delà de 2 Mo, Werkzeug refuse → 413 → envoi_trop_volumineux().
- 1 Un fichier ? extraire(fichier.filename, fichier.stream). Sinon request.form["texte"]. **Le fichier l'emporte.**
- 2 ExtractionError → page(erreur=str(echec)) : message affiché tel quel.
- 3 Trop long (> MAX_TEXT_LENGTH) ou vide → message, retour anticipé.
- 4 build_document(texte_brut, langue="fr") puis run(document).
- 5 page(), le **rendu unique** : surligner() + resumer() → render_template("analyze/index.html").

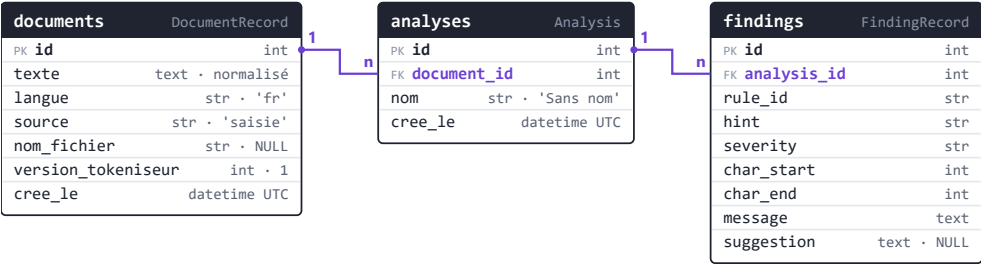
Codes HTTP : 200 ; 302 après chaque écriture ; 400 si le champ caché a été modifié à la main ; 404 pour un id inconnu ; 413 au-delà de 2 Mo.

ENREGISTRER, PUIS RELIRE

- Enregistrer** — le formulaire renvoie le texte normalisé dans un champ caché, **jamais les signalements**. Le serveur refait build_document() et run() : on ne croit jamais des positions venues du navigateur. Le navigateur renvoie les retours à la ligne en CRLF, et normalize() les ramène à LF.
- POST-Redirect-GET** : après l'écriture, un 302 vers /analyses/<id>. Rafraîchir la page refait un GET, pas un deuxième enregistrement.
- Relire** — lire_analyse(id) → build_document(analyse.document.texte, langue=analyse.document.langue). Grâce à l'idempotence, on retrouve le même texte, donc les mêmes positions → surligner() avec les FindingRecord. Même noms de champs que Finding : le gabarit ne voit pas la différence (*duck typing*).
- Messages flash** : rangés dans la session, c'est-à-dire un cookie signé avec SECRET_KEY, puis affichés une fois par base.html. Deux catégories : succes et erreur.

LE SCHÉMA — CINQ TABLES, DEUX FAMILLES SANS LIEN ENTRE ELLES

ANALYSES ENREGISTRÉES — écrites seulement sur « Enregistrer »



LISTES DE MOTS — lues à chaque analyse



PK clé primaire · **FK** clé étrangère · trait violet : relation **1 → n** (un document a plusieurs analyses ; une analyse, plusieurs signalements ; une liste, plusieurs entrées) · « NULL » : colonne facultative · après « · » : type et valeur par défaut.

LIRE LE SCHÉMA, ET CE QU'IL FAUT SAVOIR EN DIRE

Deux familles indépendantes. Un signalement enregistré garde **son propre texte** (message, suggestion) au lieu de pointer vers une entrée de liste. Supprimer « afin de » de la liste ne modifie donc aucune analyse déjà enregistrée.

char_start / char_end : les empan, des index dans `documents.texte`, le texte **normalisé**. C'est tout ce qu'il faut pour refaire le surlignage.

Ce qu'on ne stocke pas : le HTML, les tokens, l'analyse spaCy. Tout se recalcule depuis `texte` ; `version_tokeniseur` dira un jour quel tokeniseur avait produit les positions.

`cascade="all, delete-orphan"` est déclarée **dans l'ORM** (la relationship SQLAlchemy), pas en SQL (`ON DELETE CASCADE`). Supprimer une liste via SQLAlchemy supprime ses entrées. Aucune route ne supprime encore d'analyse : la cascade est prête.

Clés étrangères et SQLite : elles sont déclarées, mais SQLite ne les vérifie que si on active `PRAGMA foreign_keys = ON`, ce que le projet ne fait pas. Tout passe par l'ORM, qui remplit les clés lui-même.

UniqueConstraint : la base refuse elle-même une liste en double par langue, et une expression en double dans une liste. `ajouter_entree()` vérifie avant, pour afficher un message plutôt qu'une erreur SQL.

cree_le en UTC, via `default=maintenant` (la fonction, pas son résultat) ; seule la date est affichée.

QUOI, OÙ, AVEC QUOI

SQLite, un seul fichier : `instance/analyseur.db` (`DATABASE_URL` dans `.env`). Les tests utilisent `sqlite://`, une base en mémoire. **Flask-SQLAlchemy** fournit l'ORM, en style SQLAlchemy 2 : `Mapped[int]`, `mapped_column()`, `relationship()`.

QUI ÉCRIT, QUI LIT

Tables	Écrites par	Lues par
documents	POST /analyses/ →	GET /analyses/ (la liste)
analyses	<code>enregistrer_analyse()</code> : un	GET /analyses/<id>
findings	<code>commit</code> , trois tables	(la relecture)
word_lists	<code>flask seed</code> → <code>amorcer_liste()</code>	les règles, à chaque analyse →
word_entries	l'écran /listes/ : ajouter, supprimer	<code>lire_liste()</code>
		GET /listes/

Ⓐ Les règles LISENT, à chaque analyse

```
ConnecteursLourds.check()
→ lexiques.connecteurs_lourds("fr")
→ repositories.lire_liste("connecteurs_lourds", "fr")
→ SELECT expression, remplacement
   FROM word_entries JOIN word_lists
    WHERE nom = ... AND langue = ...
→ {"afin de": "pour", ...}
```

Pas de cache : une entrée ajoutée sur `/listes/` vaut dès l'analyse suivante. Une liste absente rend {}, et la règle ne s'applique pas, sans `KeyError`.

Ⓑ L'utilisateur ÉCRIT, sur demande

```
repositories.enregistrer_analyse(nom, document, findings,
                                nom_fichier=nom_fichier)
# Analysis(document=DocumentRecord(...),
#           findings=[FindingRecord(**asdict(f)) ...])
# → db.session.add() puis un seul commit
```

CONSTRUIRE ET REMPLIR LA BASE

`flask db upgrade` crée la **structure** : Flask-Migrate (Alembic) applique les migrations dans l'ordre.

```
7b70f94273c6 documents, analyses, findings
346c000e5787 word_lists, word_entries
1436e85304b1 analyses.nom # server_default 'Sans nom'
```

`flask seed` charge le **contenu** (16 connecteurs, 17 verbes) depuis `data/seeds/lexiques.json`. Il est idempotent : il ajoute ce qui manque et n'écrase rien.

Frontière : tout le SQL est dans `repositories.py` (9 fonctions au nom métier). Les règles lisent la base, donc elles ont besoin d'un contexte Flask : en test, c'est la fixture `base_amorcee`.

LE MOTEUR

```
REGLES: list[Rule] = [
    LongueurPhrase(), ConnecteursLourds(), Passif(),
]
regles(langue) # filtre par sapplique_a(langue)
run(document) # check() de chacune → trié par (start, end)
```

Des instances, pas des classes : une règle n'a aucun état sur `self`, la même instance sert toutes les requêtes. **Tri par position** : le surlignage parcourt le texte une seule fois, de gauche à droite. **Seuils** dans `seuils.py`, lexiques en base, par langue ; `.get()` rend `None` ou `{}` pour une langue non couverte. `LongueurPhrase` et `ConnecteursLourds` s'appliquent « là où un seuil ou un lexique existe » ; `Passif` déclare `langues = ("fr",)`.

LE RENDU HTML : SURLIGNER()

1 On fait la liste des frontières : 0, la fin du texte, et chaque `start` et `end` de signallement.

2 Pour chaque segment entre deux frontières : `escape(texte[debut:fin])`.

3 Les signalements qui couvrent tout le segment donnent `<mark class="signalement r-passif ..." data-findings="0,3">`.

4 Le résultat est un Markup : Jinja sait qu'il est sûr et ne l'échappe pas une deuxième fois.

On échappe avant de baliser, jamais l'inverse : sinon on échapperait ses propres balises, et `< → <` ; change les longueurs. Quand une phrase longue contient un connecteur, on obtient un segment à deux classes, **pas de <mark> imbriqués**. Un `<script>` collé dans le texte s'affiche tel quel.

JavaScript, 40 lignes : un clic sur un `<mark>` lit `data-findings` et allume la fiche `#signalement-N` ; un clic sur une fiche fait défiler jusqu'au premier `<mark>` correspondant. Sans JavaScript, la page reste complète.

L'IMPORT DE FICHIERS

```
REGISTRE: dict[str, Extracteur] = {
    ".txt": extraire_txt, ".md": extraire_md,
    ".docx": extraire_docx, ".odt": extraire_odt,
}
extraire(nom_fichier, flux) # par extension ; rend du brut
```

.txt	UTF-8 d'abord (BOM retiré) ; sinon <code>charset-normalizer</code> limité à 7 encodages européens ; si rien n'est sûr → refus plutôt que du « Å© »
.md	décodé comme un .txt, puis les marques sont retirées (titres, puces, citations, liens, emphase, code)
.docx	python-docx, un paragraphe par ligne ; le texte des tableaux est ignoré
.odt	odfpy, un générateur récursif garde l'ordre de lecture des titres et des paragraphes
.pdf .doc	refusés avec un message dédié (<code>FormatNonSupporte</code>)

NOTIONS PYTHON À SAVOIR EXPLIQUER

@dataclass(frozen=True)

Génère `__init__`, `__repr__` et `__eq__`. `frozen` interdit la modification : une position ne peut pas être déplacée en route.

Finding vs FindingRecord

Le premier transporte : pur Python, toujours créé. Le second stocke : une ligne SQL, créée seulement si on enregistre. Ils ont les mêmes champs, d'où `FindingRecord(**asdict(finding))`.

Rule(ABC) + @abstractmethod

Un contrat : impossible d'instancier une règle qui n'a pas de `check()`. `signaler()` reporte automatiquement l'id, le principe et la sévérité.

Literal["info", "avertissement"]

Annonce les deux seules valeurs permises. Indication pour le lecteur et l'éditeur, Python ne la vérifie pas.

@cache sur modele()

Le modèle `spaCy` est chargé une fois, au premier appel, sans `ner` (les entités nommées sont inutiles ici).

modele().pipe(textes) + next(analyses)

Toutes les phrases sont analysées en un lot. `build_document()` consomme ensuite un itérateur, phrase après phrase, dans le même ordre.

Générateur (yield, yield from)

Dans `odt.py` : il parcourt l'arbre XML en rendant les textes au fil de l'eau.

Extracteur = fonction, règle = classe

L'extracteur n'a rien à porter ; la règle porte un id, un principe, une sévérité et des langues. `Extracteur = Callable[[BinaryIO], str]`.

Liste vs dictionnaire

`REGLES` est parcourue en entier ; `REGISTRE` est indexé par extension, on y cherche un seul extracteur.

if TYPE_CHECKING:

Import pour les annotations seulement. Il casse le cycle `rules → lexiques → repositories → rules`.

create_app(), fabrique d'application

Une application neuve par test, avec `TestConfig` et une base en mémoire, sans toucher la vraie. Les extensions se branchent par `init_app()`.

Contexte d'application

`db.session` n'existe que dans une requête ou dans `with app.app_context():`. Sinon : « Working outside of application context ».

default=maintenant, pas default=maintenant()

On passe la fonction, appelée à chaque insertion. L'appeler figerait l'heure au moment de l'import.

Idempotence

`normalize(normalize(t)) == normalize(t)`, et `flask seed` relancé n'ajoute rien.

Tuple

Séquence non modifiable ; `tuple[str, ...]` veut dire « autant de `str` qu'on veut ».

ARCHITECTURE

Couches qui ne descendent que vers le bas (web → services → règles → données) : chaque morceau se teste seul. Les règles ne connaissent pas Flask, les routes ne font pas de linguistique.

Fabrique `create_app()`, extensions branchées par `init_app()` : une application neuve par test.

Deux blueprints, `analyze` et `admin` : l'analyse et l'administration sont séparées ; les URL se construisent par leur nom.

Configuration par variables d'environnement (`.env`, modèle `.env.example`) : aucun secret dans le code ; `TestConfig` hérite de la configuration et ne change que deux valeurs.

TEXTE ET POSITIONS

Un seul point d'entrée, `build_document()`, **qui normalise une fois** : un seul texte de référence, dans lequel pointent toutes les positions.

Positions reportées par addition, jamais cherchées : `find()` se trompe sur une phrase répétée.

Paragraphes d'abord, puis `pysbd` avec `clean=False` et `char_span=True` : `pysbd` ne voit qu'un bloc et rend des positions sans réécrire le texte.

Tokenisation `\S+`, grossière et assumée : elle suffit à compter les mots ; l'analyse fine vient de `spaCy`, à part.

Dataclasses figées : personne ne peut décaler une position après coup.

SPACY

`linguistics.py`, **seule frontière avec `spaCy`**, rend des `TokenLinguistique` maison : changer de bibliothèque ne toucherait que ce fichier.

`spaCy` analyse les phrases de `pysbd`, en un seul lot : une seule définition de « phrase », gouverneur reste un index valide, et le traitement va plus vite.

Modèle chargé une fois, sans `ner_fr_core_news_sm`, pas `md` : comparé sur les phrases de référence, aucun gain.

MOTEUR DE RÈGLES

Rule **abstraite + signaler()** : une règle ne peut pas oublier son id ni son principe.

Registre REGLES écrit en clair, décorateur retiré : tout ce que le moteur exécute se lit d'un coup d'œil, sans import à effet de bord.

Seuils et lexiques hors du code, par langue : jamais de `KeyError`.

Passif = dépendances + heuristiques : mesuré, de 2/6 à 4/6. **Pas de score global** : on montre l'endroit du problème, pas une note arbitraire à optimiser.

RENDU HTML

Échapper chaque segment, puis baliser : pas d'injection, pas d'entité & coupée en deux. **Découper aux frontières de tous les signalements** : un HTML valide même quand les signalements se chevauchent.

`page()`, rendu unique, gestionnaire 413 compris : le surlignage et le résumé sont calculés à un seul endroit.

IMPORT DE FICHIERS

Même motif que les règles : un contrat dans `base.py`, un registre, une fonction par format. Deux axes d'extension, une seule idée à expliquer.

`base.py` n'importe rien du projet : c'est ce qui casse le cycle `registry → docx → registry`.

`ExtractionError` au message lisible. Le `try` n'entoure qu'une ligne : une vraie erreur de programmation n'est pas masquée.

.pdf et .doc refusés avec un message dédié : le PDF n'a pas de paragraphes fiables, et le .doc est un format binaire obsolète.

Le nom de fichier du client ne sert qu'à choisir l'extracteur : il n'est jamais utilisé comme chemin sur le disque.

PERSISTANCE

`Finding (transport) ≠ FindingRecord (table)`, mêmes champs : le moteur reste testable sans base, et la traduction tient en une ligne.

Repository : tout le SQL dans `repositories.py`.

« Analyser » **n'enregistre rien** : on enregistre à la demande, sous un nom, pour ne pas encombrer la base d'essais.

On stocke le texte normalisé et les empan, jamais le HTML : une seule vérité. Un HTML relu comme sûr serait une faille, et les classes CSS ne restent pas figées.

L'enregistrement relance l'analyse côté serveur : on ne croit jamais des positions venues du navigateur.

Listes de mots en base, source versionnée `lexiques.json`, flask seed idempotent : git suit le JSON, relancer n'écrase rien, et il n'y a aucune requête réseau.

Passage en base sans toucher aux règles : `connecteurs_lourds()` et `verbes_conjugues_avec_etre()` ont gardé leur signature.

`UniqueConstraint`, cascade, version_tokeniseur : la base refuse les doublons, supprimer une analyse supprime ses signalements, et un futur tokeniseur ne cassera pas les anciennes positions.

WEB

POST-Redirect-GET après chaque écriture : rafraîchir la page ne rejoue pas l'écriture.

Retours anticipés dans `index()` : le code se lit de haut en bas. **Le fichier l'emporte sur la zone de texte**, parce que déposer un fichier est le geste le plus délibéré.

`MAX_FORM_MEMORY_SIZE` aligné sur 2 Mo : Flask 3.1 plafonne à 500 000 octets les champs non-fichier. Sans cet alignement, un texte trop long recevrait un 413 brut au lieu du message du projet.

Suppression par POST ; saisie des listes normalisée et en minuscules : pas de suppression accidentelle, et une apostrophe courbe collée correspond quand même.

Ni CSRF ni authentification : application locale et mono-utilisateur. Flask-WTF réglerait le CSRF, mais ce serait une dépendance de plus.

QUESTIONS PROBABLES

« est publiée » : pourquoi 321 à 332 ?

Ce sont des caractères : 285 (début de la phrase) + 36 (« est » dans la phrase) = 321, plus 11 caractères = 332, borne exclue. Voir p. 3.

Comment l'outil sait-il où finit une phrase ?

Les paragraphes d'abord (ligne vide), puis pysbd dans chaque paragraphe : il connaît les abréviations, les nombres et les guillemets du français.

Pourquoi pas Flesch ou un indice de lisibilité ?

Un indice rend un chiffre, pas un endroit. Et il compte des syllabes, ce qui se transpose mal d'une langue à l'autre.

Pourquoi une regex pour les connecteurs, spaCy pour le passif ?

Une locution figée est une suite de mots ; le passif est une structure. Chaque outil là où il est nécessaire.

Et fr_core_news_md ?

Comparé sur les phrases de référence : aucun gain. Le petit modèle suffit.

Un <script> collé dans le texte ?

Échappé segment par segment, affiché tel quel.

Un fichier de 50 Mo ?

Refusé par MAX_CONTENT_LENGTH avant la route ; app_errorhandler(413) rend la même page, via page().

Pourquoi le formulaire ne renvoie-t-il pas les signalements ?

On ne se fie jamais à des positions venues du navigateur : le serveur relance l'analyse.

Que contient la base, exactement ?

Le texte normalisé, les empan et les messages, plus les listes de mots. Aucun HTML : il est refait à chaque lecture.

Une liste modifiée vaut-elle tout de suite ?

Oui. Les règles lisent la base à chaque analyse, sans cache ni redémarrage.

Pourquoi le JSON et la base ?

Le JSON est la source versionnée (git ne suit pas la base) ; flask seed reconstruit la base sans rien écraser.

Pourquoi SQLite ?

L'application est locale et mono-utilisateur : un fichier, rien à installer. Pour un autre moteur, SQLAlchemy ne demanderait en principe qu'un autre DATABASE_URL.

Un .txt en cp1252 ?

UTF-8 d'abord, sinon charset-normalizer sur 7 candidats ; si rien n'est sûr, refus plutôt que du mojibake.

Pourquoi refuser le PDF ?

Il n'a pas de paragraphes fiables : les coupures feraient mentir la segmentation, les positions et le surlignage.

Ajouter l'anglais ?

Un seuil dans seuils.py, un lexique dans le JSON, une ligne dans REGLES et l'import de en.py pour les règles propres à l'anglais.

LIMITES ASSUMÉES — À DIRE SOI-MÊME

- « La porte est ouverte », « Il est convaincu » : état ou passif, c'est ambigu → signalés (un xfail le documente).
- Tokenisation \S+ : la ponctuation reste collée au mot. Suffisant pour compter.
- pysbd coupe après « l'art. », « janv. », « p. ex. » : une phrase peut être coupée en deux, et donc échapper à la règle de longueur.
- Une ligne faite seulement d'espaces ne sépare pas deux paragraphes. Le compte du diagnostic est alors faux, mais aucun signallement ne change.
- « s'agissant **des** » n'est pas détecté : le lexique traite des locutions figées, pas la morphologie.
- L'enregistrement relance les règles : si une liste change entre-temps, ce qui est enregistré diffère de ce qui était affiché.
- Base migrée mais non amorcée : rien ne plante, mais les connecteurs disparaissent et le passif perd son exclusion des verbes avec être. D'où flask seed.
- Pas de CSRF ni d'authentification : application locale et mono-utilisateur.
- Les tableaux d'un .docx sont ignorés ; l'extracteur est choisi sur l'extension, pas sur les octets.
- Date d'enregistrement en UTC : seule la date est affichée.
- L'annotation de segment() est inexacte : list[list[dict]] au lieu de list[dict]. Python ne la vérifie pas, rien ne plante.

VOCABULAIRE

empan	couple (début, fin) de caractères dans le texte normalisé
token	unité découpée dans une phrase ; ici, un mot avec sa position
lemme	forme du dictionnaire : prise → prendre, allée → aller
aux:pass	auxiliaire d'un passif (« été » dans « a été prise »)
cop	copule ; repli quand spaCy n'étiquette pas aux:pass
obl:agent	complément d'agent : « par le conseil »
gouverneur	le mot dont un autre dépend dans l'arbre (token.head)
NFC	forme Unicode composée : « é » = 1 caractère, pas 2
POS VERB	catégorie exigée du participe : écarte « est susceptible »
migration	script versionné qui fait passer le schéma d'un état au suivant
ORM	des classes Python à la place de tables SQL

PASSIF : LES SIX PHRASES DE RÉFÉRENCE

Phrase	Attendu	Dépendances seules	Règle actuelle
La décision a été prise par le conseil.	passif	✓	✓
La décision a été prise.	passif	X	✓ (repli cop)
Le rapport sera publié demain.	passif	✓	✓
Elle est allée à Paris.	non	X	✓ (liste être)
La porte est ouverte.	ambigu	X	X (xfail)
Il est convaincu de son bon droit.	non	X	X
Score		2/6	4/6

LA PHRASE DE CONCLUSION

Avant : Afin d'assurer le suivi, une notice est publiée.

Après : **Pour** assurer le suivi, **le service public** une notice. — P5 (mot simple) et P8 (agent nommé).

Deux réflexes pour toutes les réponses. ① Répondre en trois temps : **ce que ça fait** → **comment** → **pourquoi ce choix**. Rattacher le « comment » au code ou à une mesure (« pysbd coupe à chaque retour à la ligne, donc... »). ② **Ne jamais minimiser une limite** (« c'est rare », « ce sera ajouté ») : la nommer, l'expliquer, dire pourquoi elle est sans danger.

1 Que fait l'application, et pourquoi pas de score sur 100 ?

« L'application reçoit un texte administratif en français et surligne, directement dans le texte, ce qui l'éloigne de **trois des dix** principes de rédaction claire de l'UE : phrases trop longues, locutions lourdes, passif sans agent. Chaque signalement donne l'endroit exact, le principe et, si possible, une reformulation. Pas de score : un score fait optimiser un chiffre, un surlignage fait relire. »

Le score serait arbitraire (combien de points vaut un passif par rapport à une phrase longue ?), *trompeur* (3 principes sur 10 : un 90/100 ferait croire le texte clair) et *inutile* (il dit combien, jamais où).

X Ne pas dire que l'outil « vérifie que le texte respecte les principes » : il **signale des obstacles**, il ne juge pas la conformité.

2 « est publiée » va de 321 à 332 : d'où viennent ces nombres ?

« Ce sont des positions **en caractères** dans le texte normalisé, pas des numéros de mots. La phrase commence au caractère 285, et "est" se trouve 36 caractères plus loin : **285 + 36 = 321**. "est publiée" fait 11 caractères, donc la fin est à **332**, exclue comme en Python. On additionne, on ne cherche jamais : `find()` tomberait sur la première occurrence d'une phrase répétée. »

Relance « et 285 ? » : le § 1 fait 283 caractères, suivis de deux `\n`.

Invariant : `document.texte[start:end] == fragment.texte`, vérifié par un test.

3 Pourquoi normaliser une seule fois, et pas dans l'extracteur ?

« La normalisation **change la longueur** du texte : `\r\n` devient `\n`, un "é" en deux caractères n'en fait plus qu'un, le BOM et la césure disparaissent. Une position calculée avant serait faussée après. On normalise donc en premier, à un seul endroit, `build_document()`, et toutes les positions pointent dans ce texte. Un extracteur rend du brut : sinon texte collé et fichier importé pourraient donner deux textes de référence. »

X Ne pas dire que « normaliser deux fois ne donnerait pas le même résultat » : `normalize()` est **idempotente**, et la relecture d'une analyse enregistrée repose justement là-dessus.

4 Que se passe-t-il quand on clique sur « Enregistrer » ?

« Le formulaire renvoie le nom et le texte normalisé, **jamais les signalements**. Le serveur relance `build_document()` et `run()`, vérifie le nom, puis `enregistrer_analyse()` écrit l'analyse, le document et les signalements en **un seul commit**, grâce à `FindingRecord(**asdict(finding))` : les deux classes ont les mêmes champs. Enfin, une redirection 302 vers `/analyses/<id>` : rafraîchir ne réenregistre pas. »

Pourquoi relancer l'analyse ? On ne se fie jamais à des positions venues du navigateur : un champ caché se modifie en deux clics.

Limite à dire soi-même : si une liste de mots change entre l'affichage et l'enregistrement, ce qui est enregistré diffère de ce qui était affiché.

5 Pourquoi « a été prise » est signalé, et pas « est susceptible » ?

« Pour spaCy, les deux phrases ont **la même structure** : un auxiliaire étiqueté `cop` et le mot qu'il gouverne. La différence est la catégorie de ce mot : *prise* est un `VERB`, *susceptible* un `ADJ`. La règle exige un verbe : c'est cette heuristique qui a supprimé les faux positifs sur les attributs. Et c'est le repli sur `cop` qui permet de voir "a été prise", que spaCy n'étiquette pas aux:pass. »

	« a été prise »	« est susceptible »
auxiliaire	été : cop	est : cop
mot gouverné	prise : VERB	susceptible : ADJ
résultat	signalé (sans agent → avertissement)	écarté

X Ne pas se contenter de « ce n'est pas un passif » : le jury veut savoir **comment le programme le sait**.

6 Comment découpez-vous en phrases, et pourquoi pas avec spaCy ?

« En deux niveaux. D'abord les paragraphes, séparés par une ligne vide. Puis pysbd découpe chaque paragraphe en phrases : il coupe après . ! ? ... et à un retour à la ligne, mais pas après "M." ou "etc.", ni dans "3.5", "C.E.E." ou entre guillemets. Position de la phrase = début du paragraphe + position dans le paragraphe. pysbd plutôt que spaCy parce que c'est **mesuré** : spaCy fusionnait un titre avec la phrase suivante. »

X Ne pas dire « pysbd est plus rapide » : spaCy tourne ensuite sur chaque phrase de toute façon, le jury le relèvera.

Limite mesurée : pysbd coupe à tort après « l'art. » ou « p. ex. », si bien qu'une longue phrase peut échapper à la règle de longueur.

7 J'ajoute « à cet égard » sur /listes/ : quand la règle en tient-elle compte ?

« Dès l'analyse suivante, sans redémarrage : il n'y a pas de cache. La règle appelle `lexiques.connecteurs_lourds("fr")`, qui passe par `repositories.lire_liste()`, le seul endroit où l'on écrit du SQL. La règle ne sait pas que la base existe, et sa signature n'a pas changé quand les listes sont passées du code à la base. »

X Ne pas dire que la liste est « importée » : cela fait penser à un **import chargé une fois au démarrage**. Elle est **relue en base à chaque analyse**.

Nuance : une analyse déjà enregistrée ne change pas, car sa relecture ne relance pas les règles. La saisie passe par `normalize()` puis en minuscules, pour correspondre au texte analysé.

8 Et si l'on colle `<script>alert(1)</script>` ?

« Le script **s'affiche comme du texte**, et les règles l'analysent comme le reste. `surligner()` découpe le texte normalisé aux positions des signalements, **échappe chaque morceau**, puis l'entoure de `<mark>`. Dans l'autre sens, on échapperait nos propres balises, et comme `<` devient `<`, les positions ne tomberaient plus au bon endroit. »

Relance Jinja : `surligner()` renvoie un Markup, qui indique à Jinja que ce HTML est déjà sûr : pas de double échappement.

X « Il ne se passe rien » est trop court : il faut expliquer **l'ordre échapper → baliser**.

9 Pourquoi refuser le PDF ? Et un .txt en cp1252 ?

« Le PDF n'a pas de paragraphes, seulement des lignes posées sur une page. Même propre, il donne un retour à la ligne à chaque ligne visuelle, des mots coupés, des numéros de page au milieu du texte. Or pysbd coupe à chaque retour à la ligne : les phrases seraient hachées et la règle de longueur ne dirait plus rien. Le PDF ne ferait pas planter l'outil, il le ferait **mentir en silence**. Le .txt en cp1252, lui, est **accepté** : UTF-8 d'abord, sinon charset-normalizer limité à sept encodages européens, et refus si le doute persiste. »

À ajouter : le refus est un **message dédié**, pas « format inconnu » ; le PDF demanderait une nouvelle dépendance, alors que les fonctionnalités sont gelées. Un scan demanderait en plus de l'OCR.

X Ne pas dire que le cp1252 est refusé : c'est un fichier de la démo (exemples/notification.txt).

10 « 3 principes sur 10, 4 phrases sur 6 : à quoi sert votre outil ? »

« Il cible les trois défauts les plus fréquents du texte administratif : phrases trop longues, locutions lourdes, passif qui cache qui agit. Pour chacun, il montre l'endroit exact et propose souvent une reformulation. Il ne remplace pas la relecture, il la guide. Sur les dix principes, cinq relèvent du jugement du rédacteur, et deux sont des pistes que l'architecture permet d'ajouter : une classe et une ligne dans REGLES. »

Sur le 4/6 : les deux échecs sont **ambigus** (« est ouverte » : état ou action subie ? un linguiste hésite aussi). Ce sont des **faux positifs**, l'erreur sans danger : le rédacteur relit et décide. Six phrases ne font pas une statistique, c'est un **banc d'essai** qui mesure l'effet de chaque heuristique (2/6 → 4/6).

X Ne pas dire « cas rares » : « le dossier est complet », « le demandeur est tenu de » sont fréquents. Ne pas promettre « ce sera ajouté » : dire ce que ça coûterait.